Use Jupyter Lab

Use Jupyter Lab

1. Jupyter Lab installation
    1.1. Jupyter Lab
    1.2. Node.js
2. Jupyter Lab startup
    2.1. Set the default browser
    2.2. Start Jupyter Lab
    2.3. Host access
3. Jupyter Lab configuration
    3.1. LAN access
        3.1.1. Create the configuration file
        3.1.2. Modify the configuration file
    3.2. Configure the access password
    3.3. Auto-start the service at boot
        3.3.1. Edit the service file
        3.3.2. Set up the auto-start service
            Auto-start the service at boot
            Start the service
            Check the service status
            Verify the auto-start
4. Use Jupyter Lab
    4.1. Kernel
    4.2. Run the program
        4.2.1. Running
        4.2.2. Run completed

> **pip installation notes:** Under PEP 668, Ubuntu 24.04 marks the system Python as "externally managed", so installing directly with pip may report `externally-managed-environment`. `--break-system-packages` allows pip to install packages in that environment; virtual environments usually do not need this parameter, but it is kept here so that the instructions remain consistent throughout this tutorial.

> The factory system already has the relevant environment set up, so the installation steps below can be skipped.

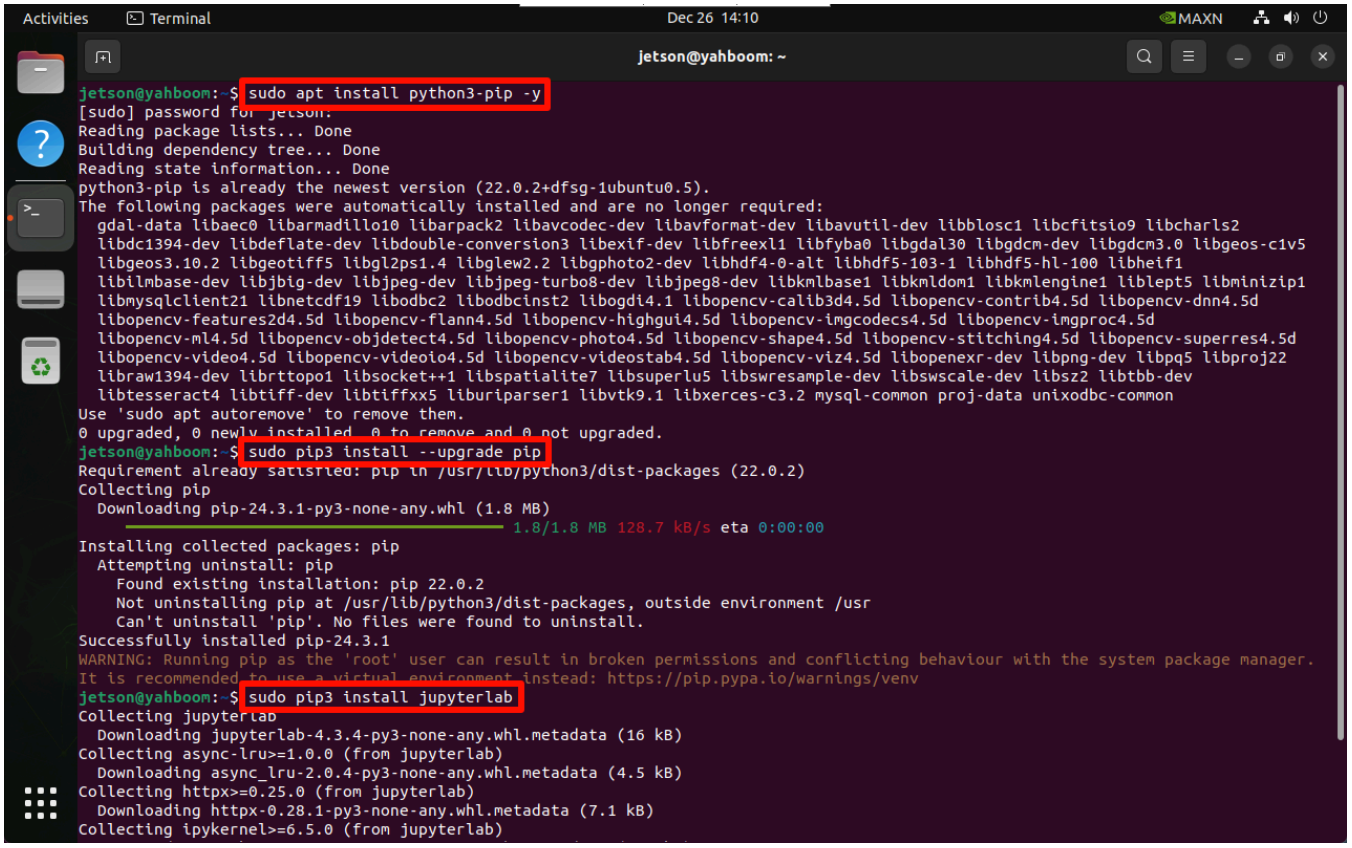
1. Jupyter Lab installation

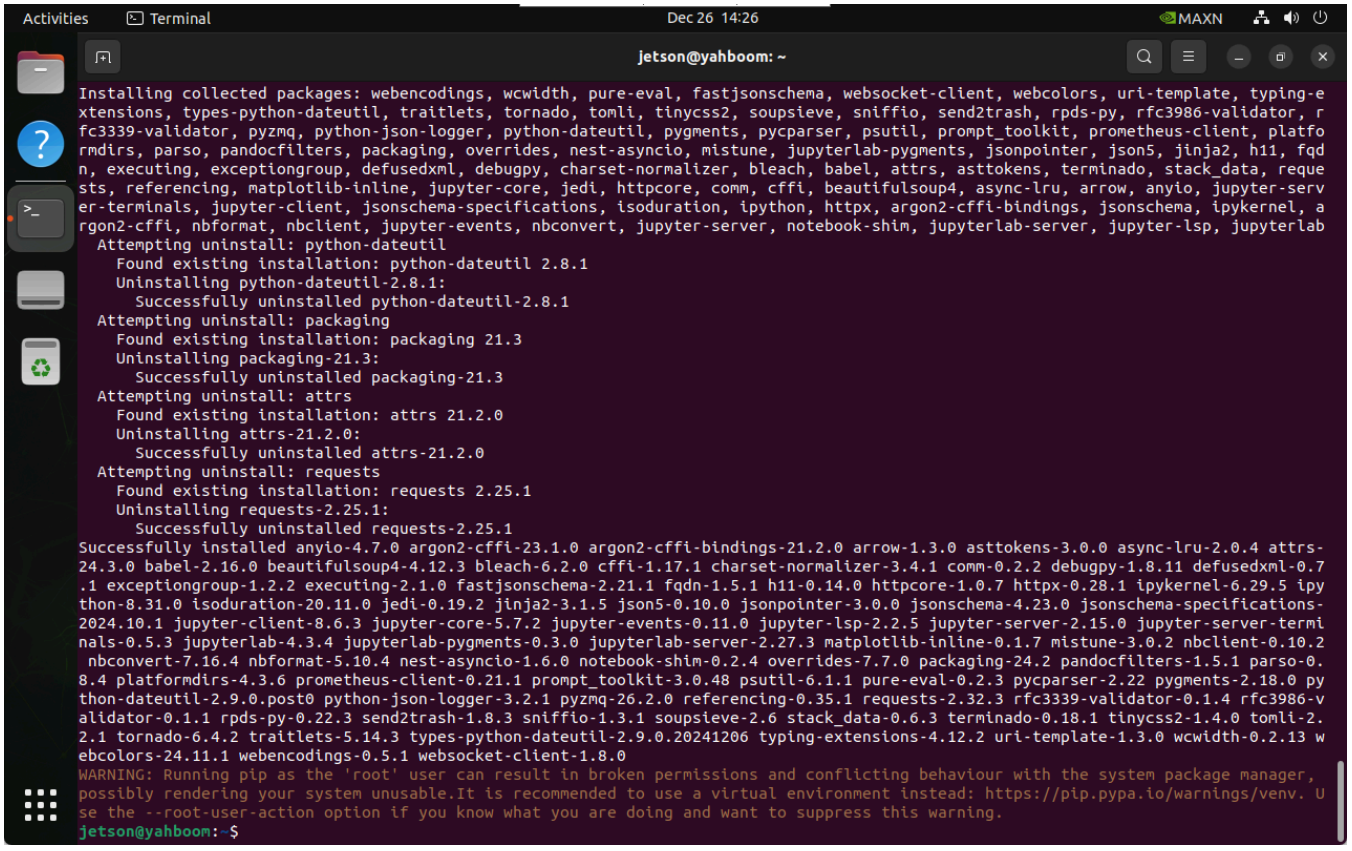
1.1. Jupyter Lab

Use the following commands to install Jupyter Lab: if the download of Jupyter Lab is slow, you can install it with a specified mirror.

```
sudo apt update
sudo apt install python3-pip -y
sudo pip3 install --upgrade pip --break-system-packages
```

```
sudo pip3 install jupyterlab --break-system-packages
# Tsinghua mirror: pip3 install jupyterlab -i https://pypi.tuna.tsinghua.edu.cn/simple --break-system-packages
# Alibaba Cloud mirror: sudo pip3 install jupyterlab -i https://mirrors.aliyun.com/pypi/simple/ --break-system-packages
```





1.2. Node.js

Use the following commands to install the latest Node.js:

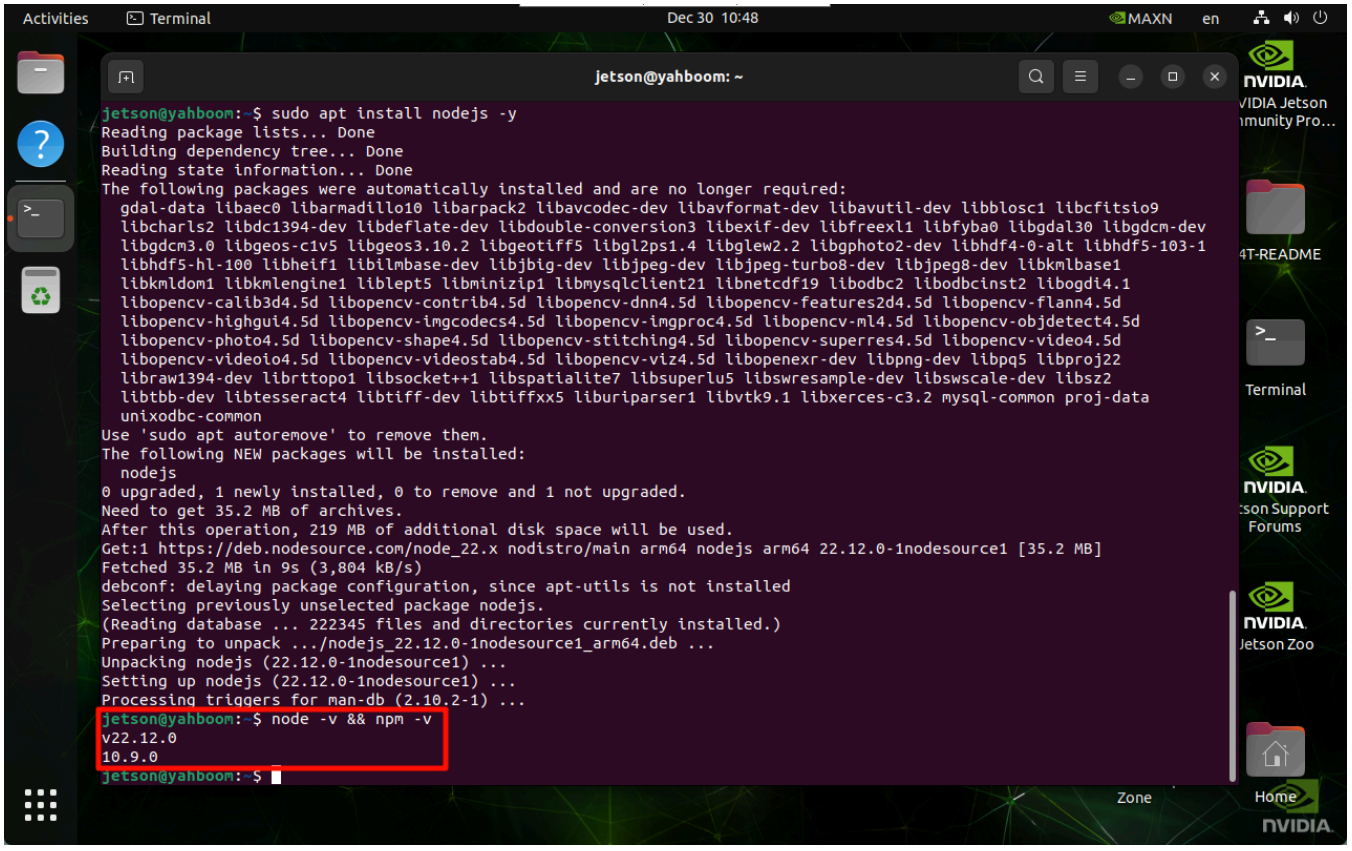
```
sudo apt install curl -y
```

```
sudo curl -fsSL https://deb.nodesource.com/setup_22.x | sudo -E bash -
```

```
sudo apt install nodejs -y
```

Verify the version:

```
node -v && npm -v
```

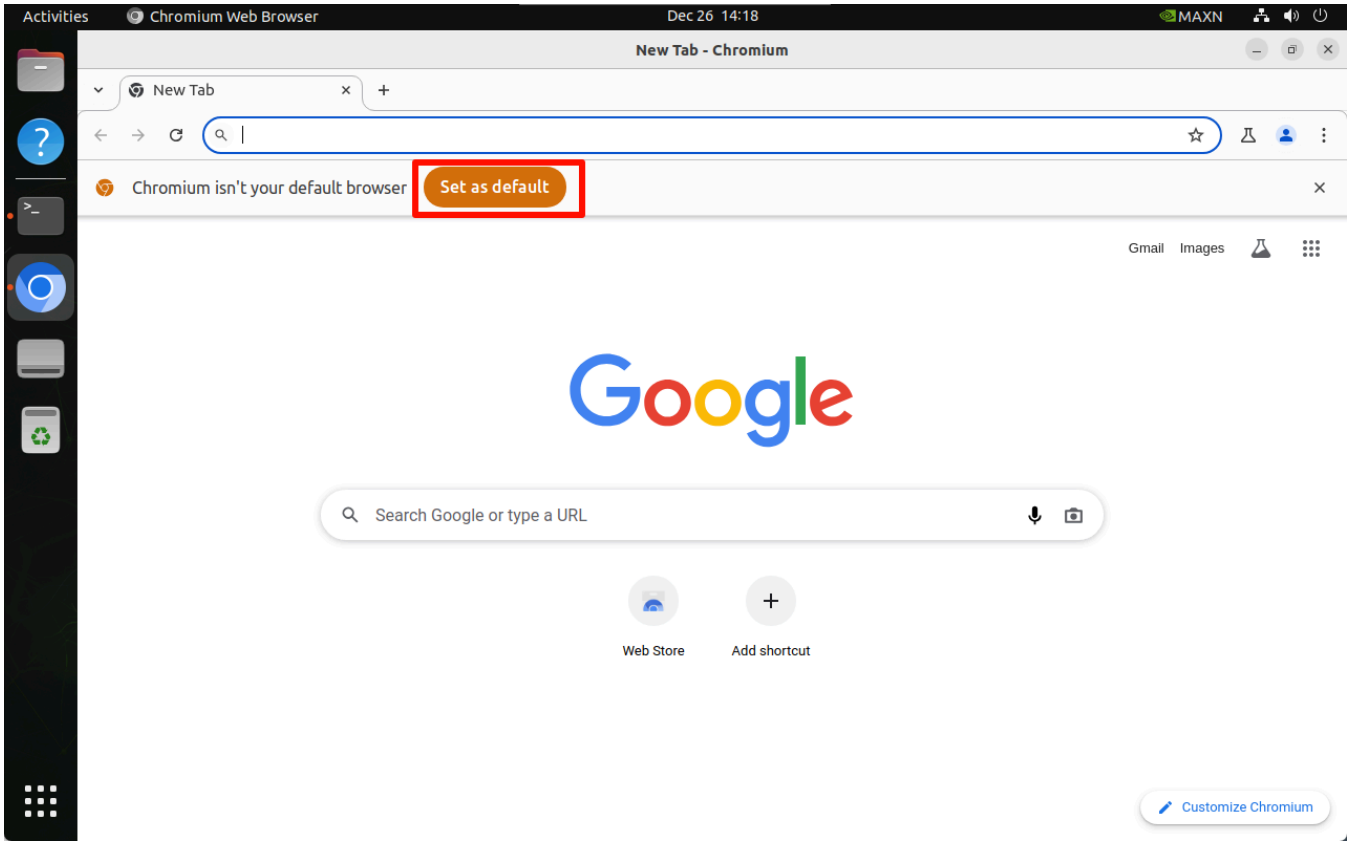


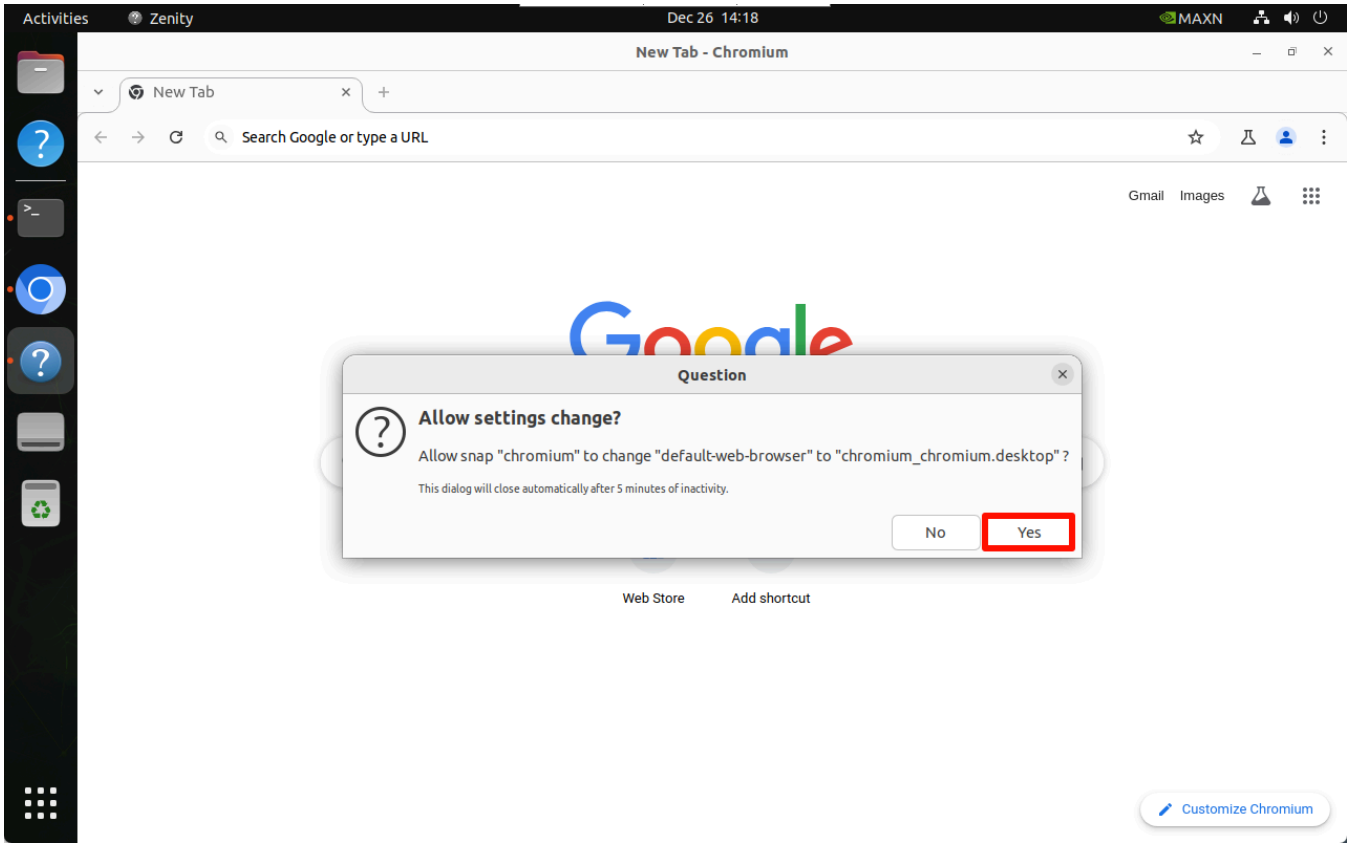
2. Jupyter Lab startup

Before starting Jupyter Lab, you need to set the system default browser, otherwise some prompts will appear when the terminal starts.

2.1. Set the default browser

Open the system Chromium browser and choose to set the default browser:

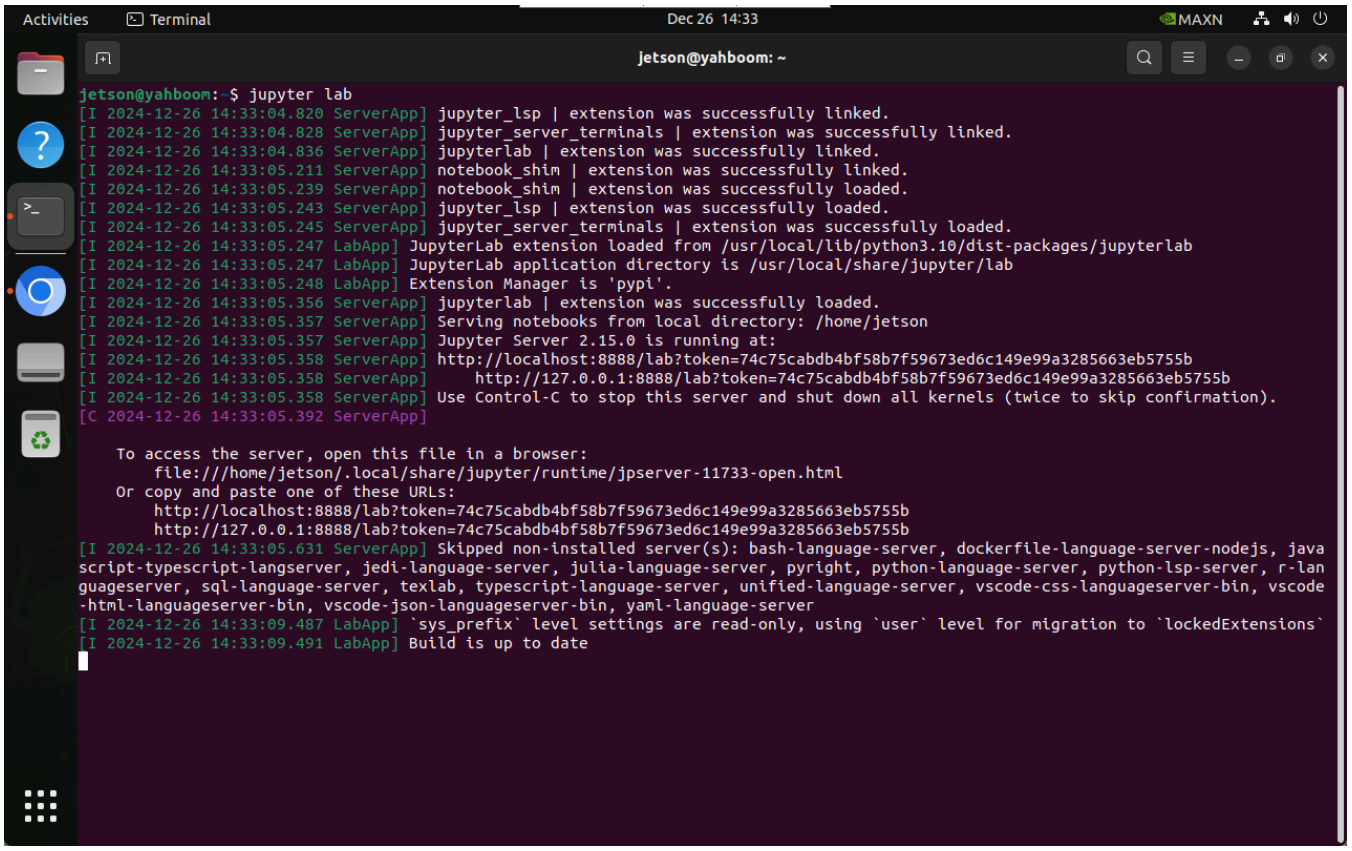


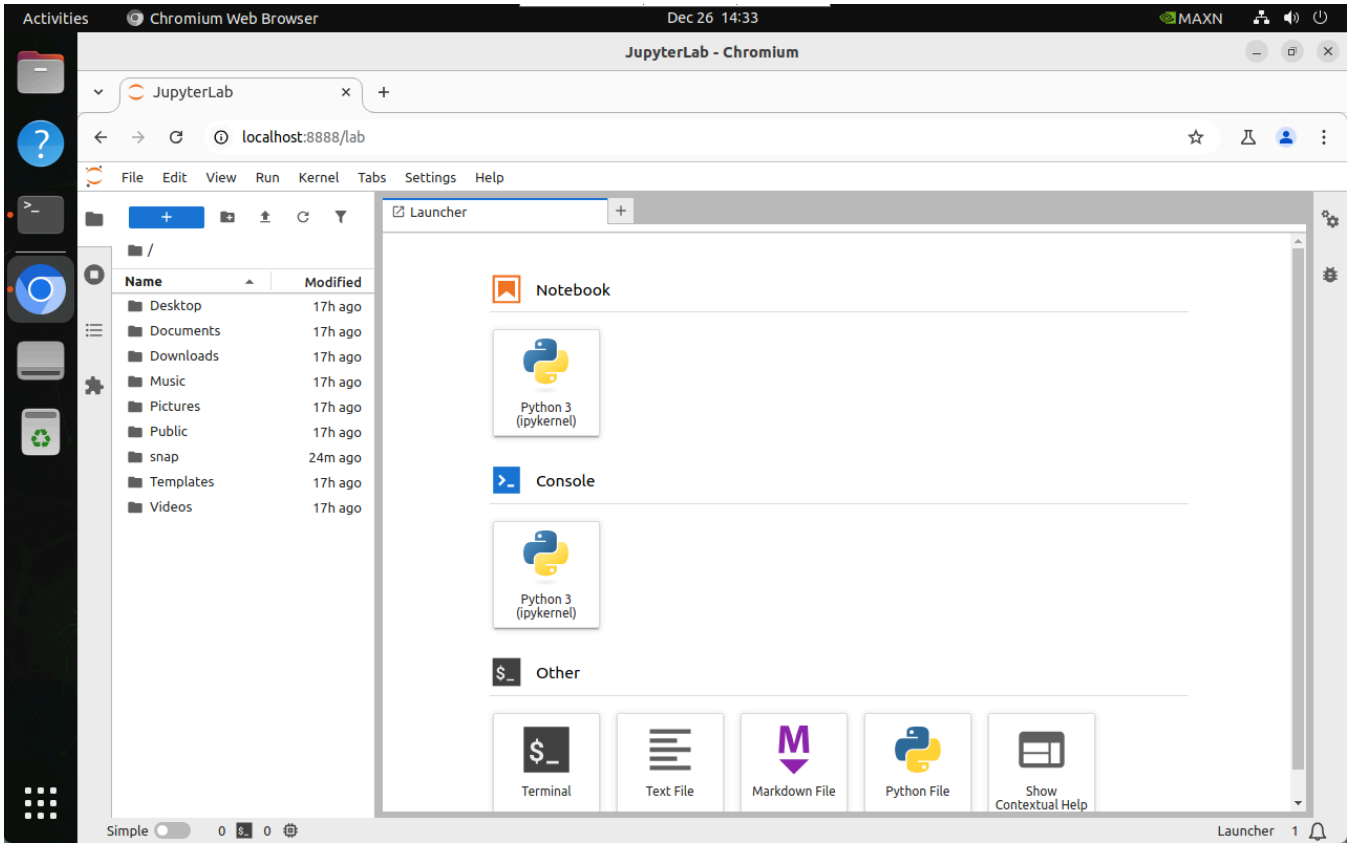


2.2. Start Jupyter Lab

> The factory system is already configured to start Jupyterlab automatically at boot, so you can jump directly to 2.3.

```
jupyter lab
# Start without a browser jupyter lab --no-browser
# Start as administrator sudo jupyter lab --allow-root
```

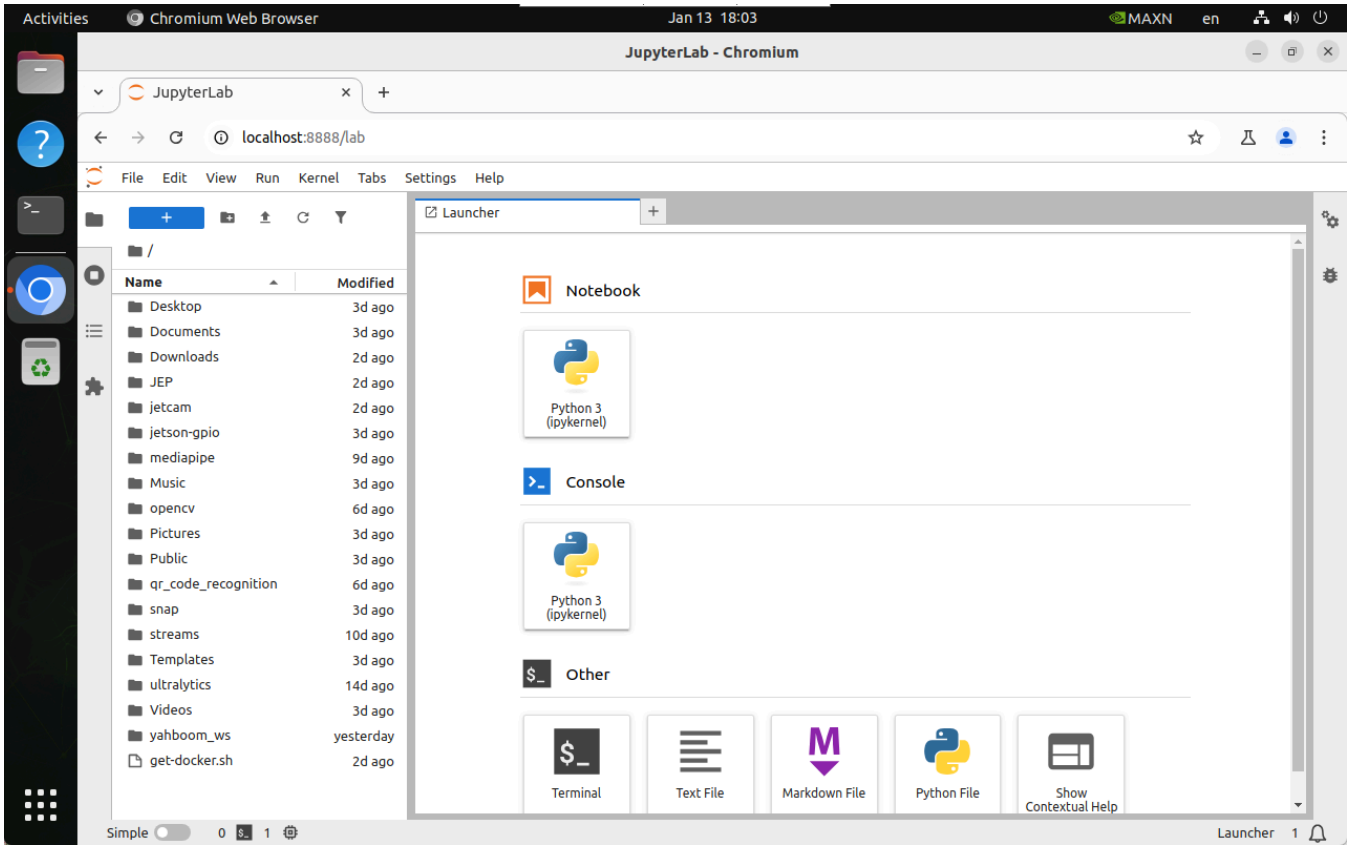




2.3. Host access

The host refers to access from the Jetson mainboard system itself, which can be reached directly through http://localhost:8888/:

```
http://localhost:8888/
```



3. Jupyter Lab configuration

Configure LAN access, the access password, auto-start at boot and so on for Jupyter Lab.

3.1. LAN access

A device on the same LAN can access it by entering IP:8888 in the browser!

Note: The LAN of a campus network is generally not reachable; you can switch to a laptop/mobile phone hotspot to test

```
For example, the mainboard IP: 192.168.2.114; you can enter 192.168.2.114:8888 in the
browser of any device on the same LAN to open the mainboard's Jupyter Lab
```

3.1.1. Create the configuration file

```
sudo jupyter lab --generate-config
```

The location of the automatically generated configuration file: Writing default config to: /root/.jupyter/jupyter_lab_config.py

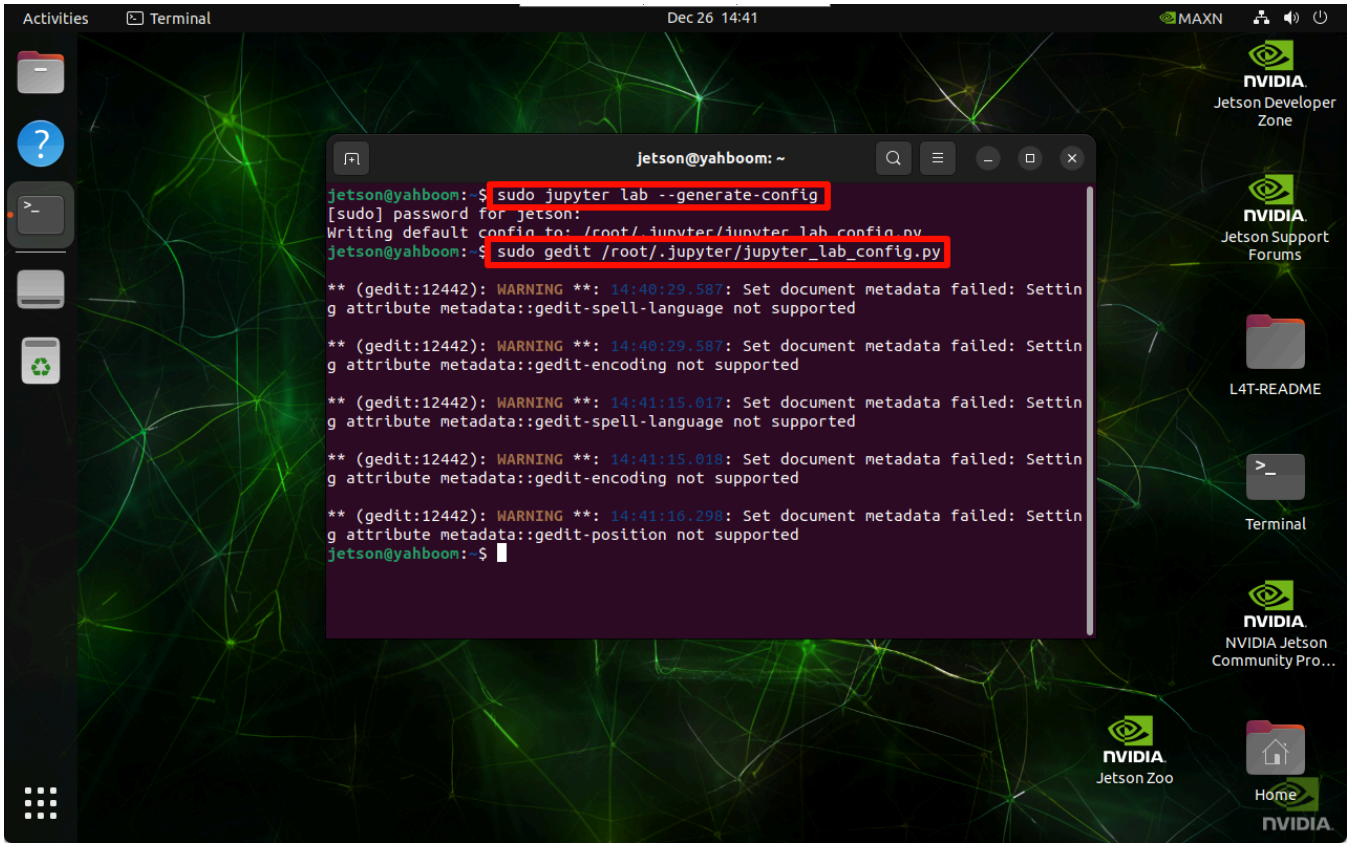
3.1.2. Modify the configuration file

```
sudo gedit /root/.jupyter/jupyter_lab_config.py
```

Content to modify: after modifying it, click Save and close the file

Operations that can be done with ctrl+F can be found and modified quickly; you can also simply copy and paste this to the bottom

```
# Allow requests from any origin to access the Jupyter Lab server
c.ServerApp.allow_origin = '*'
# 0.0.0.0 means binding all available network interfaces and allows access from any address
c.ServerApp.ip = '0.0.0.0'
# Allow the Jupyter Lab server to be started as the root user
c.ServerApp.allow_root = True
# Modify the default port to avoid conflicts
c.ServerApp.port = 8888
# Force the automatically generated Token to be cleared, preventing the Token login box from popping up first
c.ServerApp.token = ''
# Clear the automatically generated default password configuration and let the password set manually by us take over
c.ServerApp.password = ''
# Force authentication (password) to be required for login; access without a password is not allowed
c.ServerApp.allow_login_without_authentication = False
# Keep the cross-site request forgery (XSRF) check enabled to ensure the security of password login
c.ServerApp.disable_check_xsrf = False
```

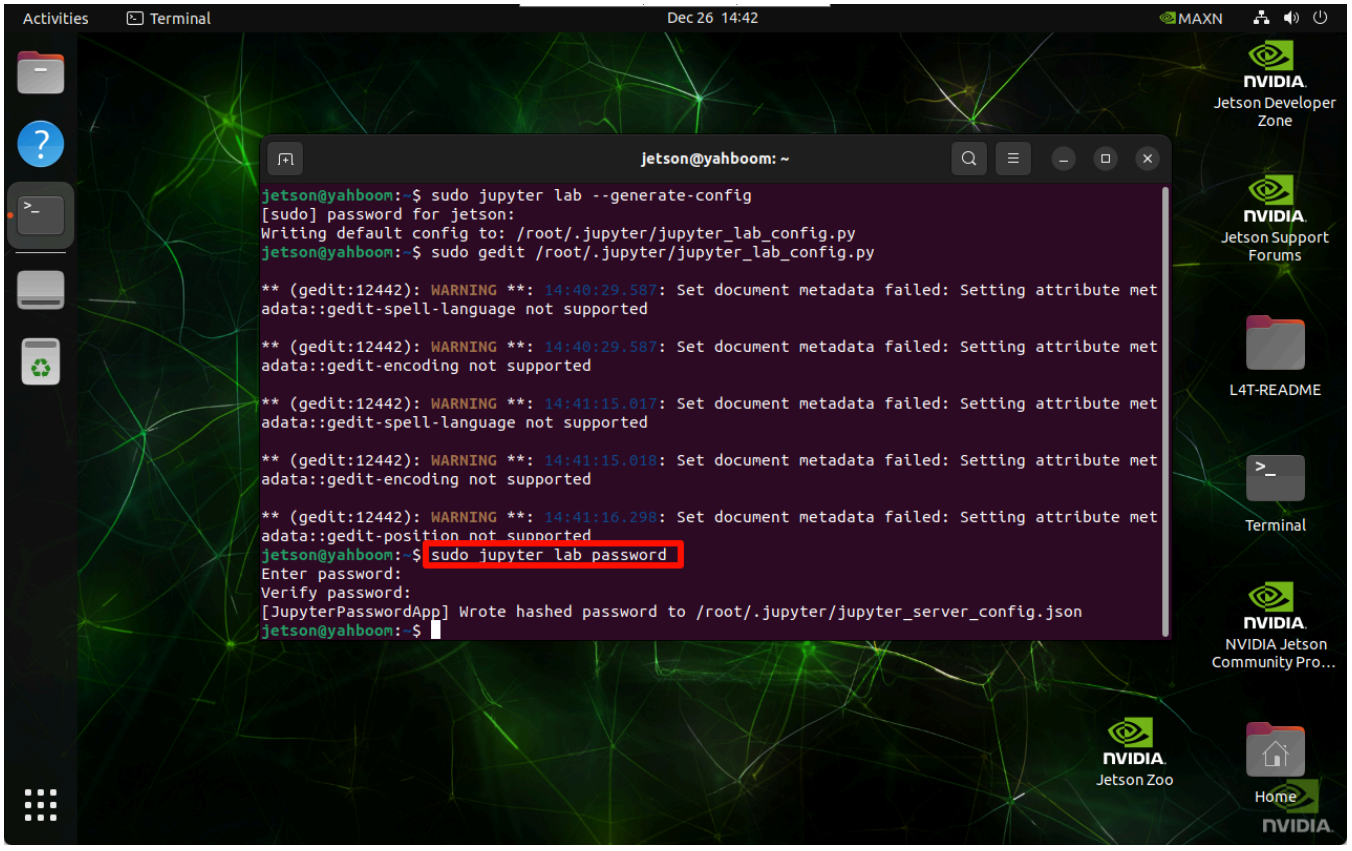


3.2. Configure the access password

Enter the command to set the password in the terminal; it has to be entered twice, and the input is not displayed while you type the password!

```
sudo jupyter lab password
```

The location of the automatically generated configuration file: [JupyterPasswordApp] Wrote hashed password to /root/.jupyter/jupyter_server_config.json



3.3. Auto-start the service at boot

3.3.1. Edit the service file

```
sudo gedit /etc/systemd/system/jupyterlab.service
```

Content to add: after adding it, click Save and close the file

```
[Unit]
Description=jupyterlab
After=network.target
[Service]
Type=simple
ExecStart=/usr/local/bin/jupyter-lab
config=/root/.jupyter/jupyter_lab_config.py --no-browser
User=root
Group=root
WorkingDirectory=/home/jetson/
```

```
Restart=always
RestartSec=10
[Install]
WantedBy=multi-user.target
```

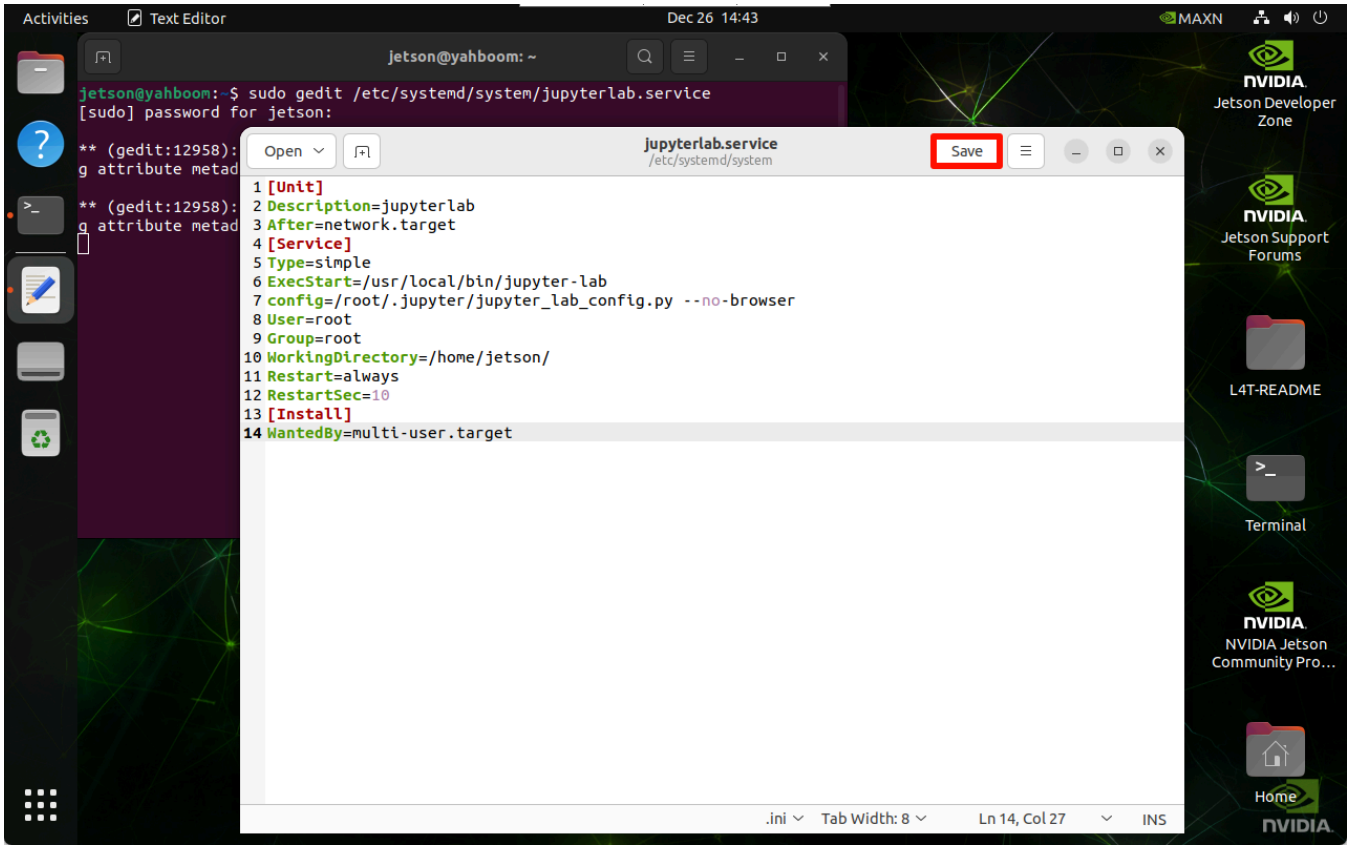
root: the system user name

ExecStart: the command that starts Jupyter lab; change it to the JupyterLab installation path

config: change it to the path of the JupyterLab configuration file

WorkingDirectory: the working directory opened when Jupyter-lab starts; you can change it yourself (it is recommended to change it to the user directory)

```
Check the Jupyter-lab installation path: which jupyter-lab
Configuration file path: refer to the path of the configuration file generated above
```



3.3.2. Set up the auto-start service

Auto-start the service at boot

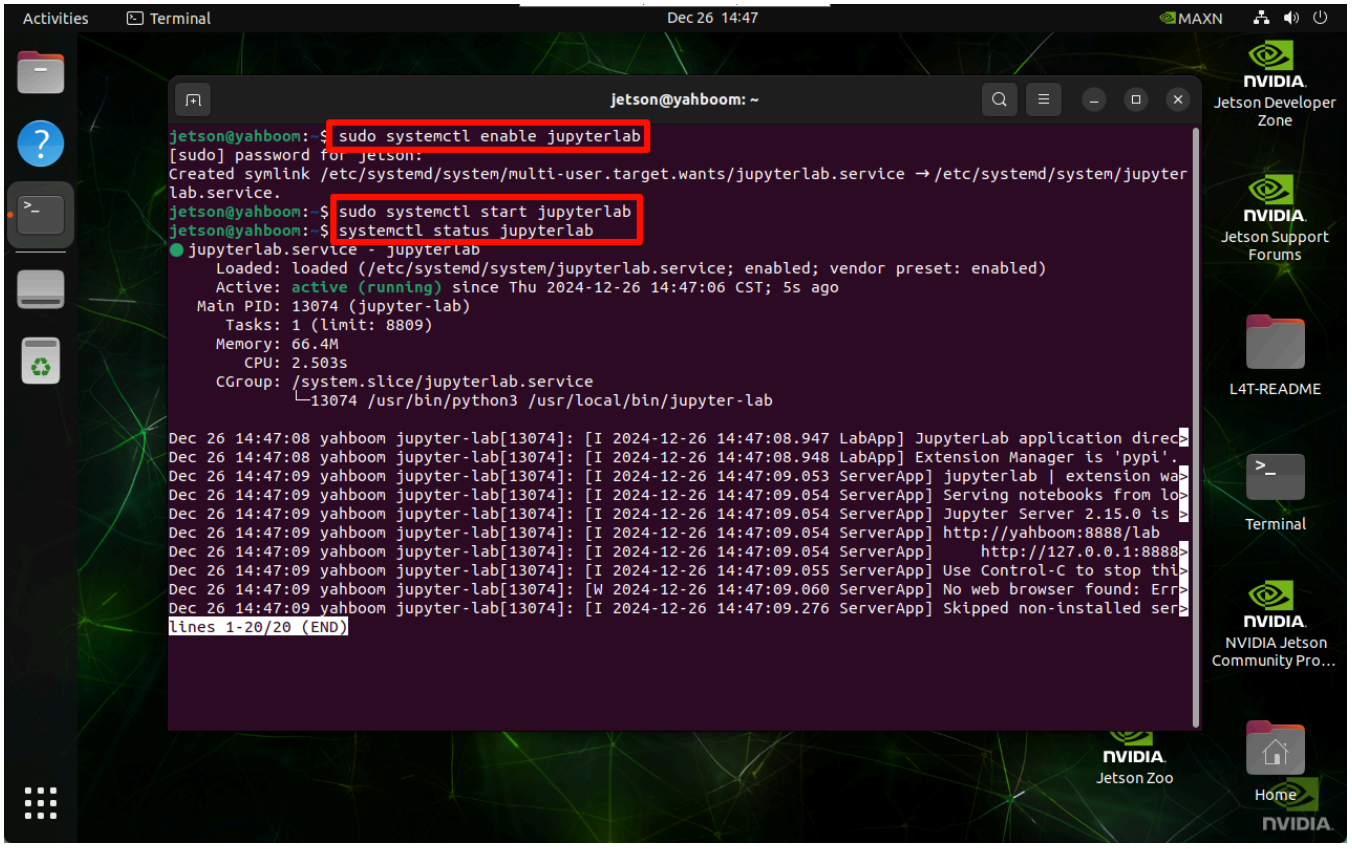
```
sudo systemctl enable jupyterlab
# Disable auto-start systemctl disable jupyterlab
```

Start the service

```
sudo systemctl start jupyterlab
# Stop the service sudo systemctl stop jupyterlab
```

Check the service status

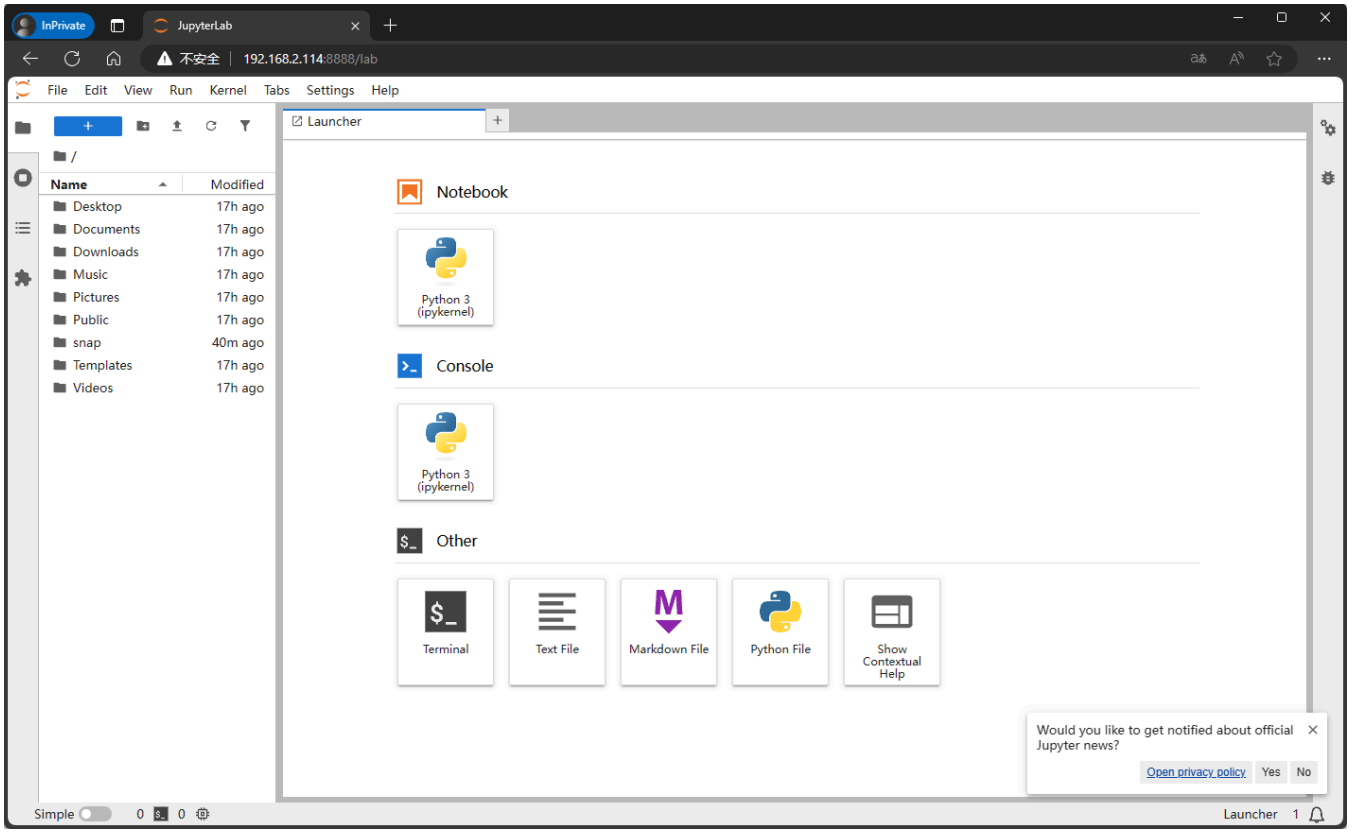
```
systemctl status jupyterlab
```



Verify the auto-start

After restarting the system, use a device on the same LAN to access mainboardIP:8888 according to the system IP.

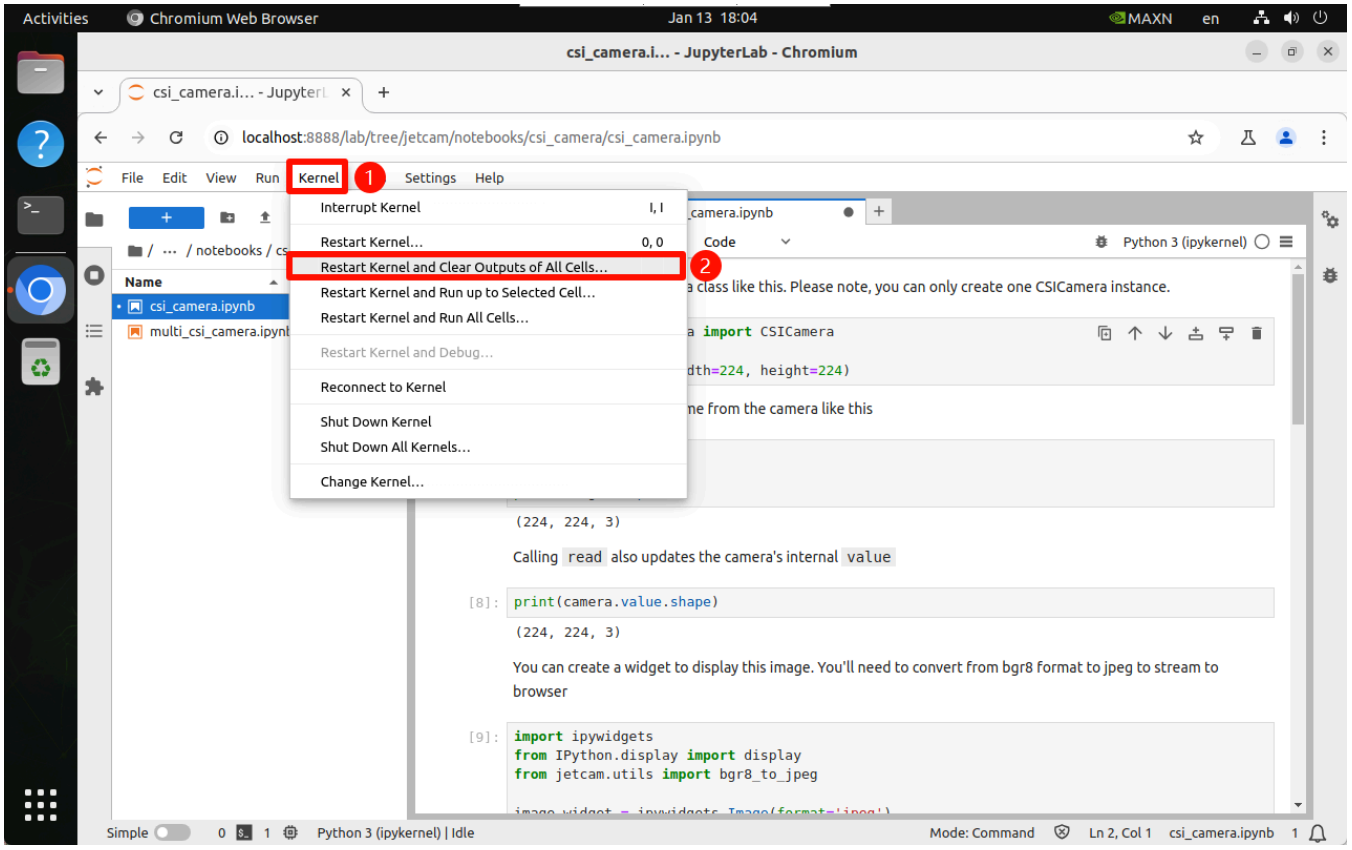
```
The password is required on the first access; it is the one set in the previous steps;
the mainboard IP in the screenshot is 192.168.2.114, so devices on the same LAN can access
192.168.2.114:8888
```



4. Use Jupyter Lab

4.1. Kernel

It is recommended to restart the kernel and clear the output of all cells every time you run a program or when the program behaves abnormally:

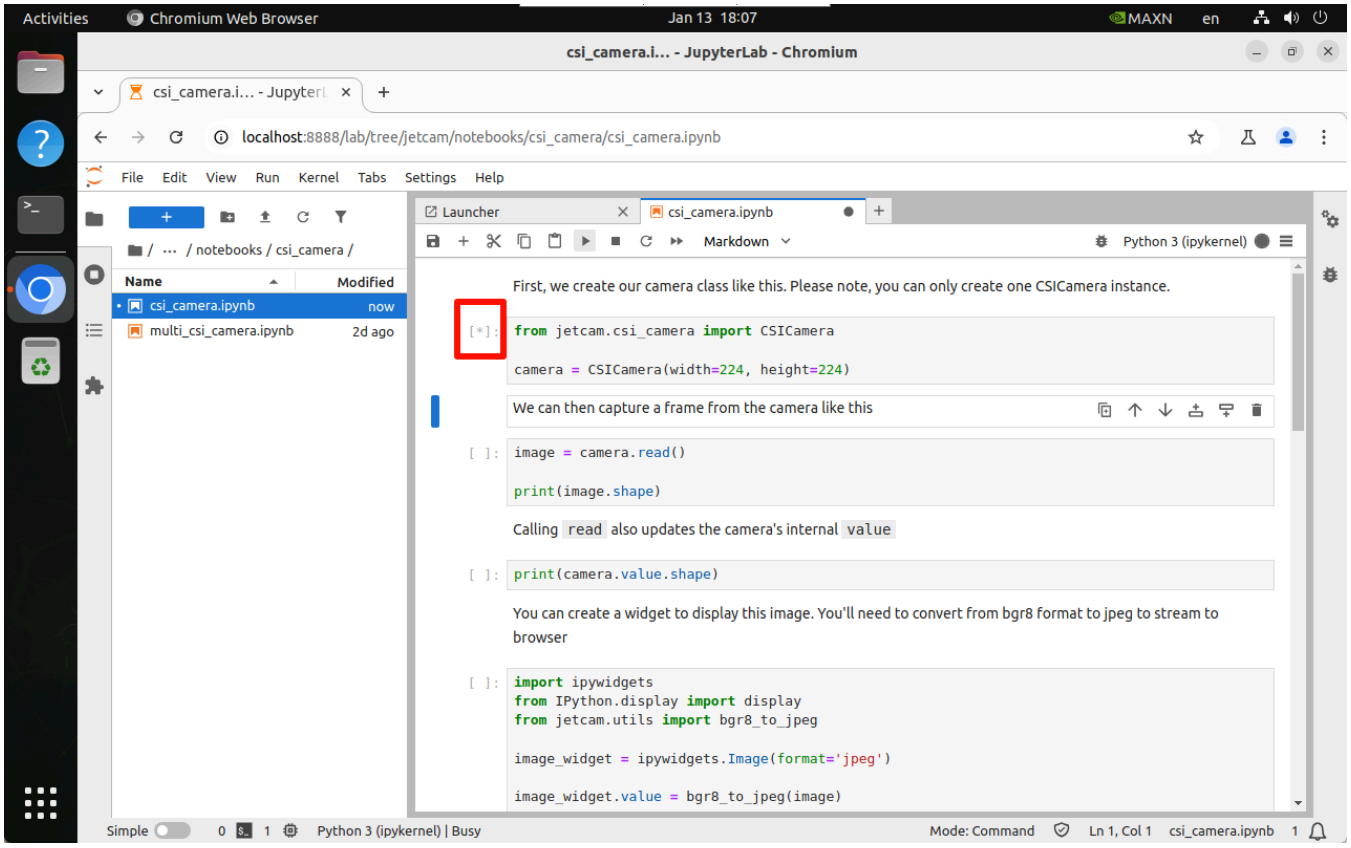


4.2. Run the program

Open the program file that you want to run through Jupyter Lab, and run the program by executing the cells from top to bottom:

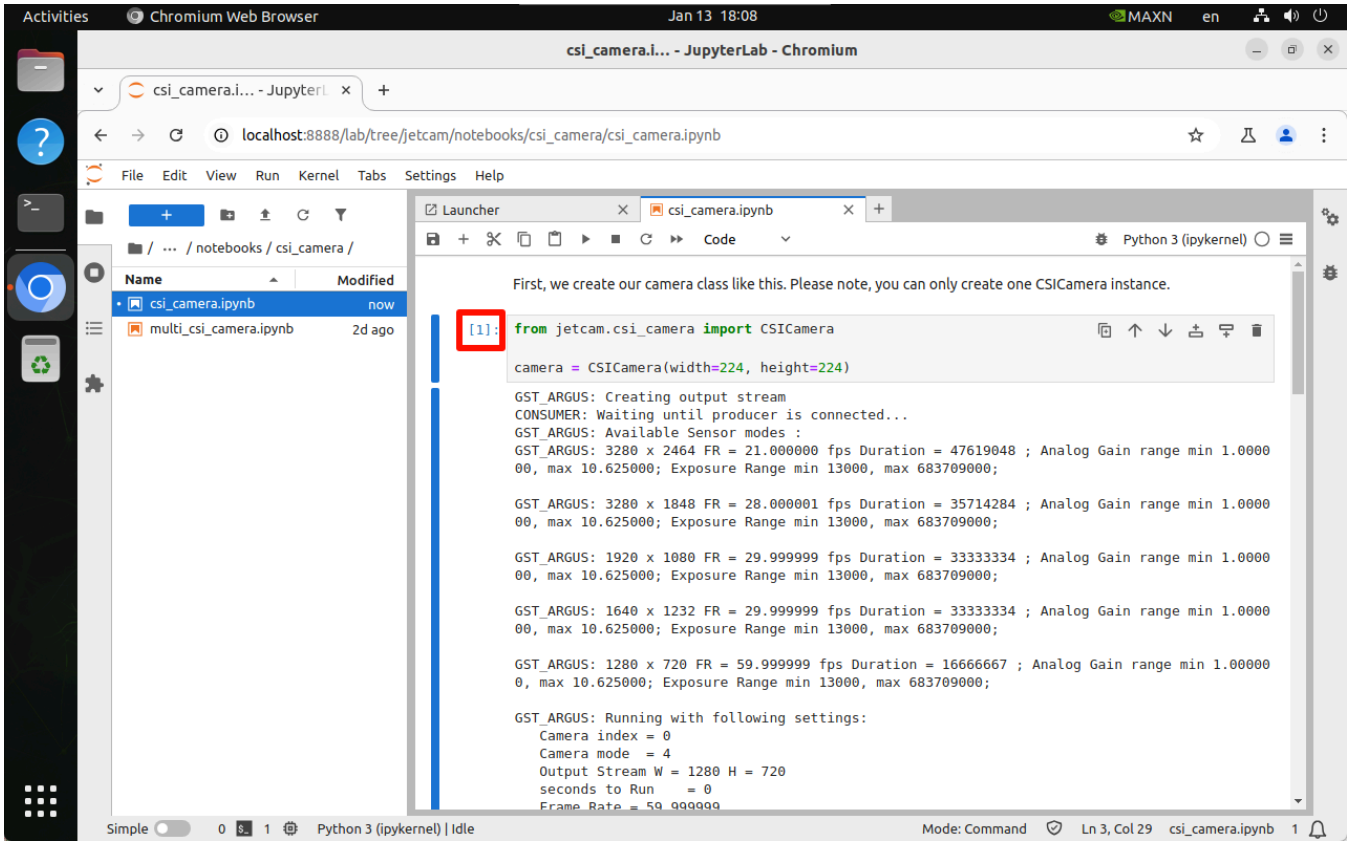
4.2.1. Running

[*] displayed in the upper left corner of the cell means it is running:



4.2.2. Run completed

[Number] displayed in the upper left corner of the cell means the number of the execution order: for example, [1] → the program ran the code of that cell for the first time


Activities
Chromium Web Browser
Jan 13 18:08
MAXN
en
csi_camera.i... - JupyterLab - Chromium
csi_camera.i... - Jupyter
localhost:8888/lab/tree/jetcam/notebooks/csi_camera/csi_camera.ipynb
File Edit View Run Kernel Tabs Settings Help
/ ... / notebooks / csi_camera /
Name
Modified
csi_camera.ipynb now
multi_csi_camera.ipynb 2d ago
Launcher
csi_camera.ipynb
Code
Python 3 (ipykernel)
First, we create our camera class like this. Please note, you can only create one CSICamera instance.
[1]:
from jetcam.csi_camera import CSICamera
camera = CSICamera(width=224, height=224)
GST_ARGUS: Creating output stream
CONSUMER: Waiting until producer is connected...
GST_ARGUS: Available Sensor modes :
GST_ARGUS: 3280 x 2464 FR = 21.000000 fps Duration = 47619048 ; Analog Gain range min 1.0000
00, max 10.625000; Exposure Range min 13000, max 683709000;
GST_ARGUS: 3280 x 1848 FR = 28.000001 fps Duration = 35714284 ; Analog Gain range min 1.0000
00, max 10.625000; Exposure Range min 13000, max 683709000;
GST_ARGUS: 1920 x 1080 FR = 29.999999 fps Duration = 33333334 ; Analog Gain range min 1.0000
00, max 10.625000; Exposure Range min 13000, max 683709000;
GST_ARGUS: 1640 x 1232 FR = 29.999999 fps Duration = 33333334 ; Analog Gain range min 1.0000
00, max 10.625000; Exposure Range min 13000, max 683709000;
GST_ARGUS: 1280 x 720 FR = 59.999999 fps Duration = 16666667 ; Analog Gain range min 1.00000
0, max 10.625000; Exposure Range min 13000, max 683709000;
GST_ARGUS: Running with following settings:
Camera index = 0
Camera mode = 4
Output Stream W = 1280 H = 720
seconds to Run = 0
Frame Rate = 59.999999
Simple
Python 3 (ipykernel) | Idle
Mode: Command
Ln 3, Col 29
csi_camera.ipynb